Phân tích module Bloom Filter

Layer 1 – Storage & Physical Data Engine

Mục lục

1 Phân tích nhiệm vụ của Bloom Filter trong đồ án **1**

2 Phân tích file `bloom_filter.hpp` **2**

2.1 Khai báo lớp và giao diện công khai 2

2.2 Hàm băm nội bộ 3

2.3 Thuộc tính dữ liệu 3

3 Phân tích file `bloom_filter.cpp` **4**

3.1 Khởi tạo bộ lọc 4

3.2 Thêm khóa vào Bloom Filter 4

3.3 Kiểm tra khả năng tồn tại 5

3.4 Xóa toàn bộ trạng thái 5

3.5 Sinh vị trí băm thứ n 6

4 Tóm tắt luồng hoạt động của module **7**

5 Phân tích cách triển khai **7**

1. Phân tích nhiệm vụ của Bloom Filter trong đồ án

Trong đồ án Cloud Storage ở Layer 1, Bloom Filter đóng vai trò là tầng kiểm tra sơ bộ trước khi hệ thống truy vấn vào cấu trúc lưu trữ chính xác hơn như `Hash Table` hoặc `Chunk Index`. Mục tiêu của nó không phải là xác nhận chắc chắn một chunk đã tồn tại, mà là giúp loại bỏ thật nhanh những trường hợp chắc chắn chưa từng xuất hiện.

Trong pipeline deduplication, mỗi chunk sau khi được băm sẽ tạo ra một `ChunkID`. Tại thời điểm kiểm tra trùng lặp, Bloom Filter nhận vào khóa dạng chuỗi này và trả lời theo hai khả năng:

- **Chắc chắn không tồn tại**: nếu có ít nhất một bit tương ứng chưa được bật.
- **Có thể tồn tại**: nếu tất cả các bit kiểm tra đều đã bật.

Ý nghĩa của cơ chế này trong đồ án là:

1. giảm số lần tra cứu trực tiếp vào bảng băm hoặc chỉ mục vật lý;

2. tăng tốc bước tiền xử lý khi số lượng chunk lớn;

3. giữ cho bước dedup có chi phí thấp ở giai đoạn sàng lọc ban đầu.

Bloom Filter phù hợp với kiến trúc Layer 1 vì module này chỉ cần một quyết định nhanh để chọn nhánh xử lý tiếp theo:

- nếu Bloom Filter báo *không có*, hệ thống có thể đi tiếp sang nhánh lưu chunk mới;
- nếu Bloom Filter báo *có thể có*, hệ thống mới cần dùng cấu trúc chính xác để xác minh.

Do đó, Bloom Filter là một bộ lọc xác suất nhằm tối ưu hiệu năng, chứ không thay thế cho cơ chế xác thực chính xác. Đây là điểm rất quan trọng trong đồ án: **quyền quyết định cuối cùng về dedup vẫn phải thuộc về bảng chỉ mục chính xác**. Nếu Bloom Filter dương tính giả thì chỉ làm tăng thêm một lần kiểm tra, nhưng không được phép làm mất dữ liệu hoặc gộp nhầm hai chunk khác nhau.

2. Phân tích file `bloom_filter.hpp`

2.1. Khai báo lớp và giao diện công khai

```
class BloomFilter {
public:
    BloomFilter(std::size_t bit_count = 1 << 20, std::size_t
       hash_count = 3);

    void add(const std::string& key);
    bool possibly_contains(const std::string& key) const;
    void clear();
```

File `hpp` định nghĩa giao diện của lớp `BloomFilter` theo hướng rất gọn. Nhìn vào phần `public`, có thể thấy module này cung cấp đúng ba thao tác cốt lõi mà một Bloom Filter cơ bản cần có.

Hàm khởi tạo cho phép cấu hình hai tham số chính:

- `bit_count`: số lượng bit của mảng đánh dấu;
- `hash_count`: số hàm băm được mô phỏng để ánh xạ một khóa vào nhiều vị trí.

Hai giá trị mặc định `1 « 20` và `3` cho thấy người triển khai hướng tới một phiên bản dễ dùng ngay cả khi chưa tinh chỉnh theo tải thực tế. Điều này phù hợp với đồ án vì có thể khởi tạo nhanh một Bloom Filter đủ nhẹ để tích hợp vào pipeline dedup.

Ba hàm công khai thể hiện đúng vai trò của cấu trúc:

- `add`: chèn một khóa vào bộ lọc;
- `possibly_contains`: kiểm tra khả năng tồn tại của khóa;
- `clear`: đặt lại toàn bộ trạng thái.

Thiết kế giao diện như vậy có ưu điểm là dễ hiểu, ít phụ thuộc, và rất thuận tiện khi Layer 1 chỉ cần nhúng Bloom Filter như một thành phần hỗ trợ bên cạnh bảng chỉ mục chính.

2.2. Hàm băm nội bộ

```
private:
    std::size_t nth_hash(std::size_t n, const std::string& key)
        const;
```

Phần `private` khai báo một hàm hỗ trợ tên là `nth_hash`. Đây là ý tưởng quan trọng vì Bloom Filter cần nhiều vị trí băm cho cùng một khóa. Thay vì cài nhiều bộ băm khác nhau ở giao diện bên ngoài, tác giả gom logic tạo vị trí băm vào một hàm nội bộ.

Cách tổ chức này giúp lớp giữ được tính đóng gói:

- bên ngoài chỉ cần biết cách thêm và kiểm tra khóa;
- chi tiết sinh ra vị trí bit được ẩn hoàn toàn bên trong lớp.

Về mặt thiết kế phần mềm, đây là cách tách trách nhiệm hợp lý. Người dùng lớp không phải quan tâm mỗi khóa được ánh xạ ra bao nhiêu vị trí và bằng công thức nào; họ chỉ sử dụng kết quả cuối cùng của bộ lọc.

2.3. Thuộc tính dữ liệu

```
private:
    std::size_t bit_count_;
    std::size_t hash_count_;
    std::vector<bool> bits_;
};
```

Ba thuộc tính riêng mô tả đầy đủ trạng thái của Bloom Filter:

- `bit_count_`: kích thước không gian bit;
- `hash_count_`: số lần băm mỗi khóa;
- `bits_`: vùng nhớ lưu trạng thái các bit.

Điểm đáng chú ý là tác giả dùng `std::vector<bool>`. Dù đây là một dạng chuyên biệt của `vector`, nó vẫn phù hợp với ý tưởng Bloom Filter vì cấu trúc này bản chất chỉ cần lưu các giá trị bit bật/tắt. Chọn như vậy giúp mã ngắn gọn, dễ đọc và bám sát bản chất bài toán.

Từ file `hpp`, có thể kết luận module được xây theo hướng tối giản nhưng đúng trọng tâm: giao diện nhỏ, trạng thái rõ ràng, và đủ cơ sở để phục vụ chức năng tiền kiểm tra dedup trong Layer 1.

3. Phân tích file `bloom_filter.cpp`

3.1. Khởi tạo bộ lọc

```
BloomFilter::BloomFilter(std::size_t bit_count, std::size_t
    hash_count)
    : bit_count_(bit_count), hash_count_(hash_count), bits_(
        bit_count, false) {}
```

Constructor khởi tạo trực tiếp ba thành phần dữ liệu thông qua *initializer list*. Đây là cách viết tốt trong C++ vì tránh gán lại không cần thiết sau khi đối tượng đã được tạo.

Ý nghĩa của từng phần khá rõ:

- lưu lại kích thước bộ lọc và số lượng hàm băm;
- tạo mảng `bits_` có đúng `bit_count` phần tử;
- đặt toàn bộ bit ban đầu về `false`.

Nhờ đó, Bloom Filter ngay khi tạo ra đã ở trạng thái rỗng và sẵn sàng nhận dữ liệu. Đây là điểm quan trọng vì mọi truy vấn `possibly_contains` trước khi có thao tác `add` đều phải cho kết quả âm tính.

3.2. Thêm khóa vào Bloom Filter

```
void BloomFilter::add(const std::string& key) {
    for (std::size_t i = 0; i < hash_count_; ++i) {
        bits_[nth_hash(i, key)] = true;
    }
}
```

Hàm `add` cài đặt đúng nguyên lý của Bloom Filter. Với mỗi khóa đầu vào, hàm lặp qua `hash_count_` lần và dùng `nth_hash` để sinh ra từng vị trí bit tương ứng. Mỗi vị trí sau đó được bật lên `true`.

Điểm đáng chú ý là thao tác thêm phần tử chỉ có một chiều: bit đã bật thì không tắt lại trong quá trình thêm. Điều này phản ánh đúng bản chất của Bloom Filter chuẩn:

- hỗ trợ thêm nhanh;
- không hỗ trợ xóa phần tử theo cách trực tiếp;
- chấp nhận khả năng nhiều khóa cùng ánh xạ vào một số bit giống nhau.

Trong ngữ cảnh đồ án, khi một `ChunkID` mới được xác nhận là đã xuất hiện trong hệ thống, nó có thể được đưa vào Bloom Filter để các lần kiểm tra sau diễn ra nhanh hơn.

3.3. Kiểm tra khả năng tồn tại

```
bool BloomFilter::possibly_contains(const std::string& key) const
    {
    for (std::size_t i = 0; i < hash_count_; ++i) {
        if (!bits_[nth_hash(i, key)]) {
            return false;
        }
    }
    return true;
}
```

Đây là phần trung tâm của module. Hàm kiểm tra cùng các vị trí bit đã được dùng khi thêm khóa. Nếu chỉ cần một bit chưa được bật, có thể kết luận ngay khóa đó chắc chắn chưa từng được thêm vào. Vì vậy hàm trả về `false` ngay lập tức.

Ngược lại, nếu tất cả các bit đều đang bật, hàm trả về `true`. Tuy nhiên, kết quả này chỉ mang nghĩa *có thể tồn tại*. Nguyên nhân là nhiều khóa khác nhau có thể vô tình đặt trùng lên cùng tập vị trí bit.

Cách cài đặt ở đây thể hiện rất đúng triết lý của Bloom Filter:

- **không có âm tính giả**: nếu hàm trả về `false`, khóa chắc chắn chưa được thêm;
- **có thể có dương tính giả**: nếu hàm trả về `true`, vẫn phải kiểm tra lại bằng cấu trúc chính xác.

Đối với Layer 1, đây là bước sàng lọc nhanh trước khi truy cập vào bảng quản lý chunk thật sự. Nhờ đó, module góp phần giảm số lượng truy vấn nặng hơn ở các tầng sau của pipeline dedup.

3.4. Xóa toàn bộ trạng thái

```
void BloomFilter::clear() {
    std::fill(bits_.begin(), bits_.end(), false);
}
```

Hàm `clear` đưa toàn bộ mảng bit về trạng thái ban đầu. Cách dùng `std::fill` là trực tiếp và hiệu quả cho mục tiêu này.

Về mặt ý nghĩa, đây không phải thao tác xóa một khóa đơn lẻ, mà là đặt lại toàn bộ Bloom Filter. Trong thực tế hệ thống, hàm này phù hợp với các tình huống như:

- tái khởi tạo module khi nạp lại dữ liệu từ chỉ mục chính;
- làm sạch trạng thái cho một lượt chạy kiểm thử;
- rebuild bộ lọc từ nguồn dữ liệu chuẩn.

Việc chỉ hỗ trợ `clear` thay vì `remove` cũng hợp với đặc trưng của Bloom Filter cơ bản, bởi xóa từng phần tử sẽ dễ làm ảnh hưởng tới các khóa khác có chung bit.

3.5. Sinh vị trí băm thứ n

```
std::size_t BloomFilter::nth_hash(std::size_t n, const std::
    string& key) const {
    static std::hash<std::string> hasher;
    std::string mixed = key + "#" + std::to_string(n);
    return hasher(mixed) % bit_count_;
}
```

Hàm `nth_hash` là cách tác giả mô phỏng nhiều hàm băm từ một bộ băm chuỗi chuẩn của C++. Ý tưởng ở đây là ghép thêm chỉ số `n` vào khóa để tạo ra các chuỗi khác nhau, từ đó sinh ra nhiều giá trị băm khác nhau.

Ví dụ, cùng một khóa nhưng với `n = 0`, `n = 1`, `n = 2`, hệ thống sẽ băm các chuỗi khác nhau như:

- `key#0`
- `key#1`
- `key#2`

Sau khi băm, kết quả được lấy modulo `bit_count_` để rơi vào miền chỉ số hợp lệ của mảng `bits_`.

Cách triển khai này có ưu điểm là đơn giản và dễ cài đặt. Nó tránh việc phải mang vào nhiều thuật toán băm phức tạp, đồng thời vẫn đáp ứng được yêu cầu cốt lõi của một Bloom Filter ở mức đồ án. Đối với hệ thống học thuật hoặc prototype, đây là lựa chọn hợp lý vì ngắn gọn và đủ minh họa bản chất thuật toán.

4. Tóm tắt luồng hoạt động của module

Luồng làm việc của module có thể tóm tắt như sau:

1. Hệ thống khởi tạo `BloomFilter` với số bit và số lần băm mong muốn.
2. Khi một khóa mới cần ghi nhận, hàm `add(key)` được gọi.
3. Bên trong `add`, module sinh ra nhiều vị trí bit bằng `nth_hash` và bật chúng lên.
4. Khi cần kiểm tra một khóa, hàm `possibly_contains(key)` được gọi.
5. Nếu có ít nhất một bit chưa bật, module trả về `false`; khi đó có thể kết luận khóa chắc chắn chưa có trong bộ lọc.
6. Nếu tất cả các bit đều đã bật, module trả về `true`; khi đó hệ thống phải chuyển sang bước kiểm tra chính xác bằng bảng băm hoặc chỉ mục chunk.
7. Khi cần tái thiết lập toàn bộ trạng thái, hàm `clear()` sẽ đưa tất cả bit về `false`.

Đặt vào đồ án Cloud Storage, luồng này thường nằm giữa bước sinh `ChunkID` và bước tra cứu chính xác trong `chunk index`. Vì vậy Bloom Filter hoạt động như một *cổng lọc nhanh*, giúp dedup chạy hiệu quả hơn khi số lượng chunk tăng dần.

5. Phân tích cách triển khai

Nhìn tổng thể, cách triển khai module này là **gọn, đúng bản chất, và phù hợp với phạm vi đồ án**. Tác giả không cố làm Bloom Filter quá phức tạp, mà tập trung vào những thao tác quan trọng nhất: khởi tạo, thêm khóa, kiểm tra khóa, và làm sạch trạng thái. Điều này giúp mã nguồn dễ đọc, dễ kiểm thử, và rất thuận lợi khi tích hợp vào Layer 1 như một thành phần hỗ trợ dedup.

Một số điểm làm tốt có thể ghi nhận:

- giao diện lớp nhỏ và rõ ràng, nên rất dễ sử dụng từ module khác;
- constructor dùng *initializer list*, thể hiện cách viết C++ sạch;
- phần kiểm tra và chèn bám đúng nguyên lý Bloom Filter;
- ý tưởng sinh nhiều giá trị băm từ một hàm băm chung giúp mã ngắn và dễ hiểu.

Tuy nhiên, vẫn có một vài hạn chế nhỏ cần nhìn ra:

1. `std::hash<std::string>` không phải lúc nào cũng là lựa chọn mạnh nhất nếu muốn kiểm soát tốt chất lượng phân bố bit giữa các môi trường hoặc giữa các lần chạy khác nhau;

2. module hiện chưa có cơ chế đo hoặc ước lượng tỷ lệ dương tính giả, nên khó tinh chỉnh `bit_count` và `hash_count` theo quy mô dữ liệu thực tế;

3. cấu trúc này chỉ hỗ trợ `clear` toàn bộ, chưa hỗ trợ các biến thể như *counting bloom filter* nếu sau này cần xóa phần tử linh hoạt hơn.

Nhìn chung, với vai trò là một module Bloom Filter trong đồ án DSA về Cloud Storage, cách triển khai này là **đủ tốt để minh họa đúng tư tưởng dedup tăng tốc bằng bộ lọc xác suất**. Nó thể hiện được ý chính của bài toán, giữ được mã nguồn sáng sủa, và phù hợp để làm nền trước khi mở rộng lên các phiên bản tối ưu hơn.